Master-Seminar

Construction Theory of Information und Software in the Natural World

Aufgabenblatt 2 – Team 2

Makrozustände, Abstandszonen und Attractor Memory

Fokus

Schrittweise Ableitung robuster Makrozustände aus den mikrophysikalischen Simulationen von Team 1. Im Mittelpunkt stehen physikalisch begründete Abstandszonen, Kontaktgraphen, Persistenzfilter, Rückkehrdynamik und die Frage, wann ein Zustand wirklich ein Zustand ist.

Leitfrage

Lassen sich aus Positionen, Paarabständen und Kontaktgraphen stabile Makrozustände wie Monomer-Gas, Dimer, Trimer, Tetraeder-ähnliche Cluster oder größere Aggregate extrahieren – und zeigen diese Zustände eine Form von attractor-basierter Rückkehrdynamik?

Feld	Angabe
Team	Team 2 – Makrozustände und Attractor Memory
Zeithorizont	8 Wochen, anschlussfähig an Team 1 und Team 3
Programmiersprache	Python 3, NumPy, Pandas, NetworkX, Matplotlib
Hauptprodukte	Kontaktzonen, Kontaktgraphen, Makrozustandssequenzen, Dwell-Time- und Return-Analysen
Wichtigster Grundsatz	Ein Makrozustand ist nicht nur ein Label, sondern eine robuste, reproduzierbare und dynamisch relevante Klasse von Mikrokonfigurationen.

1. Rolle von Team 2 im Gesamtseminar

Team 1 liefert Mikrodynamik: Positionen, Kräfte, Potentiale und Paarabstände.

Team 2 übersetzt diese Mikrodaten in robuste Makrozustände.

Erst dadurch können Team 3 ε-Maschinen rekonstruieren, Team 4 Closure prüfen, Team 5 Tasks definieren und Team 8 Sustainability-Metriken berechnen.

Team 2 ist deshalb die entscheidende Brücke zwischen physikalischer Simulation und informationeller Interpretation. Wenn die Makrozustände schlecht definiert sind, werden alle späteren Analysen instabil. Wenn sie dagegen robust, reproduzierbar und physikalisch begründet sind, entsteht eine belastbare Datenbasis für das gesamte Seminar.

2. Didaktische Lernziele

Die Studierenden lernen, wie aus kontinuierlichen Mikrodaten diskrete Makrozustände entstehen. Sie verstehen, warum Coarse-Graining keine bloße Vereinfachung ist, sondern eine wissenschaftliche Modellentscheidung. Sie lernen, Abstände nicht binär, sondern in physikalisch motivierte Zonen zu klassifizieren, Kontaktgraphen zu berechnen, Flackerzustände zu filtern und Rückkehrdynamik als Hinweis auf attractorartige Stabilität zu messen.

3. Wissenschaftlicher Ausgangspunkt

Team 2 soll nicht einfach Clustergrößen zählen. Stattdessen soll es fragen: Welche mikroskopischen Konfigurationen sind auf Makroebene gleichbedeutend? Welche Zustände sind nur momentane Zufallsereignisse? Welche Zustände sind robust genug, um als Informationsträger oder Attraktor-Kandidaten weiterverwendet zu werden?

Ein Zustand wird erst dann interessant, wenn er wiedererkennbar, persistent, robust gegen kleine Störungen und dynamisch relevant ist. Dadurch entsteht die Brücke zu Attractor Memory: Das System kann nach einer Störung in eine ähnliche Makroklasse zurückkehren, auch wenn die einzelnen Mikrokonfigurationen nicht identisch sind.

Merksatz

Team 2 erzeugt nicht einfach schöne Labels. Team 2 entscheidet, welche physikalischen Mikrodaten später als Zustände, Attraktoren, ε-Maschinen-Kontexte, Closure-Variablen und Sustainability-Signaturen verwendet werden dürfen.

4. Eingabedaten von Team 1

Team 2 soll zunächst prüfen, welche Daten von Team 1 tatsächlich vorhanden sind. Mindestens erforderlich sind Positionstrajektorien. Idealerweise liefert Team 1 bereits Paarabstände und Kontaktzonen.

Spalte	Bedeutung	Status
run_id	eindeutiger Lauf / Simulation	Pflicht
seed	Zufallsstartwert	Pflicht für Reproduzierbarkeit

time oder t	Zeitschritt	Pflicht
particle_id	Protozellenindex	Pflicht
x, y, optional z	Positionen	Pflicht
model_type	vdw_only, dlvo_classic, cl_retarded, cl_meso_hypothesis	Pflicht
material_type	fatty_acid oder PMBC	sehr empfohlen
A_H / A_eff	Hamaker-/Materialparameter	sehr empfohlen
salt_mM / lambda_D	Salzkonzentration / Debye-Länge	sehr empfohlen
noise, temperature	Simulationsparameter	sehr empfohlen

Wichtig für die Teamkommunikation

Team 2 soll nicht warten, bis alle Daten perfekt sind. Beginnen Sie mit einem kleinen Testdatensatz von Team 1 oder synthetischen Beispieldaten. Entscheidend ist, dass die Auswertungspipeline früh funktioniert.

5. Schritt-für-Schritt-Anleitung

Schritt 1: Daten laden und Einheiten prüfen

Importieren Sie die Positionsdaten. Prüfen Sie, ob Längen in Meter, Nanometer oder dimensionslosen Einheiten vorliegen. Dokumentieren Sie die Umrechnung eindeutig. Einheitenfehler übertragen sich direkt in falsche Kontaktzonen.

Schritt 2: Oberflächenabstände berechnen

Berechnen Sie für jedes Paar i,j den Oberflächenabstand $L_{ij}(t)=||x_i-x_j||-2R$. Verwenden Sie nicht den Mittelpunktabstand als Kontaktabstand. Für R=500 nm ist 2R=1000 nm.

Schritt 3: Abstandszonen klassifizieren

Ordnen Sie jeden Paarabstand einer physikalisch begründeten Zone zu: hard_contact, dlvo_zone, near_gap, meso_gap, far_gap oder separated. Diese Zonen werden später an Team 3, 4 und 8 weitergegeben.

Schritt 4: Kontaktgraphen erzeugen

Erzeugen Sie pro Zeitpunkt mehrere Kontaktgraphen: G_hard, G_near und G_meso. Dadurch kann später getestet werden, ob ein Zustand nur bei sehr strenger Kontaktdefinition oder auch bei mesoskaliger Nähe stabil ist.

Schritt 5: Graphfeatures berechnen

Berechnen Sie pro Zeitpunkt largest_cluster_size, number_of_edges, number_of_components, mean_degree, cluster_size_distribution und optionale Kompaktheitsmaße.

Schritt 6: Makrozustand labeln

Leiten Sie aus Graphfeatures Makrozustände ab. Beginnen Sie mit einem kleinen Katalog: monomer_gas, dimer_dominated, trimer_like, tetra_like, large_cluster, mixed_transition.

Schritt 7: Persistenzfilter anwenden

Filtern Sie Zustände, die kürzer als min_dwell_steps auftreten. Speichern Sie raw_macrostate und filtered_macrostate. Team 3 soll bevorzugt die gefilterte Sequenz verwenden.

Schritt 8: Dwell Times messen

Berechnen Sie für jeden Zustand die Verweildauerverteilungen. Eine lange Dwell Time ist ein erster Hinweis auf Stabilität, aber noch kein Beweis für Attractor Memory.

Schritt 9: Rückkehrdynamik messen

Messen Sie Return Probability und Mean Return Time. Ein Attraktor-Kandidat liegt vor, wenn ein Zustand nach Verlassen wiederholt innerhalb eines Zeithorizonts zurückkehrt.

Schritt 10: Robustheit gegen Rauschen prüfen

Fügen Sie kleine Positionsstörungen hinzu, labeln Sie erneut und messen Sie Label Agreement. Zustände, die bei minimalem Jitter verschwinden, sind nicht robust genug.

Schritt 11: Modelle und Materialien vergleichen

Vergleichen Sie vdw_only, dlvo_classic, cl_retarded und cl_meso_hypothesis sowie fatty_acid vs PMBC. Arbeiten Sie heraus, welche Modelle persistente Dimer- oder Tetraeder-ähnliche Zustände erzeugen.

Schritt 12: Export an Team 3 vorbereiten

Speichern Sie lange, saubere Makrozustandssequenzen mit Metadaten. Team 3 benötigt stabile diskrete Sequenzen; flackernde Labels zerstören die ε-Maschinen-Rekonstruktion.

6. Physikalisch begründete Abstandszonen

Zone	Abstandsbereich	Interpretation
hard_contact	L < 2 nm	Core, Hydration, sehr nahe Membranannäherung; numerisch und physikalisch empfindlich
dlvo_zone	2 ≤ L < 5 nm	elektrostatische DLVO-Repulsion kann bei Salz noch relevant sein
near_gap	5 ≤ L < 10 nm	Übergangsbereich; Elektrostatik stark gedämpft, vdW/CL noch relevant
meso_gap	10 ≤ L < 30 nm	retardierte Lifshitz-/Materialeffekte; mögliche PMBC-Stabilisierung
far_gap	30 ≤ L < 100 nm	Einzelpaarbindung schwach; nur als mesoskalige Nähe oder Mehrkörperfenster interpretieren
separated	L ≥ 100 nm	keine Kontaktkante im Standardgraphen

7. Mehrere Kontaktgraphen statt eines einzigen Graphen

Graph	enthaltene Zonen	Nutzung
G_hard	hard_contact	strenge Nahkontaktstruktur
G_near	hard_contact + dlvo_zone + near_gap	physikalisch enger Kontaktbereich
G_meso	hard_contact + dlvo_zone + near_gap + meso_gap	mesoskalige Kopplungsstruktur bis 30 nm
G_far	bis far_gap	nur explorativ; nicht als harte Bindung interpretieren

Didaktischer Hinweis

Ein Dimer bei L=3 nm ist nicht dasselbe wie zwei Protozellen bei L=80 nm. Beide können räumlich nah sein, aber nur der erste Fall sollte als starker Kontakt gelten. Deshalb müssen Abstandszonen explizit erhalten bleiben.

8. Makrozustandskatalog

Makrozustand	Kriterium	Interpretation
monomer_gas	keine oder fast keine Kontakte	robuster Baseline-Zustand
dimer_dominated	größte Komponente = 2 oder viele Paare	wichtiger erster Clusterzustand
trimer_like	größte Komponente = 3	Zwischenzustand / kleine Clusterbildung
tetra_like	größte Komponente = 4 und kompakt genug	zentraler Kandidat für stabile Minimalcluster
loose_four_cluster	vier Komponentenmitglieder, aber zu wenige Kanten	nicht mit tetra_like verwechseln
large_cluster	größte Komponente ≥ 5	größere Aggregate, ggf. unspezifisch
mixed_transition	wechselhafte, nicht eindeutig klassifizierte Struktur	Übergangs- oder Flackerzustand
failure_disperse	nach vorherigem Cluster vollständige Zerstreuung	relevant für Team 8

Wann ist ein Zustand wirklich ein Zustand?

Ein Zustand ist nicht bloß ein Snapshot. Ein Zustand ist eine reproduzierbare, robuste und dynamisch relevante Klasse von Mikrokonfigurationen. Er muss über mehrere Zeitschritte bestehen, gegen kleine Störungen stabil sein und idealerweise Rückkehrdynamik zeigen.

9. Attractor-Memory-Metriken

Metrik	Definition	Bedeutung
Dwell Time	Länge zusammenhängender Aufenthalte in Zustand z	Persistenz
Return Probability	Wahrscheinlichkeit, nach Verlassen innerhalb T zurückzukehren	Attractor-Hinweis
Mean Return Time	mittlere Rückkehrzeit nach Verlassen	Zeitskala der Rückkehr
Self-transition Probability	P(Z_{t+1}=z | Z_t=z)	Stabilität im Zustandsraum
Transition Matrix	P(Z_{t+1}=j | Z_t=i)	Dynamik zwischen Makrozuständen
Label Agreement	Anteil gleicher Labels nach Jitter	Robustheit
Seed Reproducibility	Zustand tritt in mehreren Seeds auf	Reproduzierbarkeit

Kriterium	Vorschlag	Kommentar
Häufigkeit	> 5 % der Zeitpunkte	sonst zu selten für robuste Statistik
mittlere Dwell Time	> 5–10 Schritte	abhängig von dt und Samplingrate
Return Probability	> 0.3 innerhalb T	nur als Seminarheuristik
Seed-Reproduzierbarkeit	mindestens 3 Seeds	keine Einmal-Artefakte
Label Agreement	> 0.7 brauchbar, >0.9 sehr gut	Jitter-Robustheit

10. Vergleich der Team-1-Modelle

Modelltyp	Bedeutung	Frage an Team 2
vdw_only	nicht-retardierte Nahbereichsanziehung	Referenz: Wie viel erklärt die Hamaker-Anziehung allein?
dlvo_classic	Elektrostatik + vdW + Core	klassische Kolloid-Baseline
cl_retarded	retardierte Lifshitz-Approximation	zeigt Abschwächung gegenüber vdW im Mesobereich
cl_meso_hypothesis	hypothesengeleiteter Mesoskalen-Term	nur klar als Forschungsmodell interpretieren

Für jedes Modell sollen dieselben Metriken berechnet werden: Häufigkeit der Makrozustände, Dwell Times, Return Probability, Transition Matrix, Label Robustness und Unterschiede zwischen Fettsäurevesikeln und PMBCs.

11. Vollständig formatierter Code-Anhang

Der folgende Code ist ein ausführliches Startgerüst. Er soll nicht blind übernommen, sondern getestet, kommentiert und an die echten Team-1-Daten angepasst werden.

```
# src/zones.py
import numpy as np

R_DEFAULT = 500e-9

def surface_distance(p_i, p_j, R=R_DEFAULT, box=None):
    """
    Berechnet den Oberflächenabstand L_ij = Mittelpunktabstand - 2R.
    Optional: periodische Randbedingungen mit minimum image convention.
    """
    delta = np.asarray(p_j) - np.asarray(p_i)
    if box is not None:
        delta = delta - box * np.round(delta / box)
    center_distance = np.linalg.norm(delta)
    return center_distance - 2.0 * R


def distance_zone(L):
    """
    Klassifiziert einen Oberflächenabstand L in Meter.
    """
    L_nm = L * 1e9
    if L_nm < 2:
        return "hard_contact"
    if L_nm < 5:
        return "dlvo_zone"
    if L_nm < 10:
        return "near_gap"
    if L_nm < 30:
        return "meso_gap"
    if L_nm < 100:
        return "far_gap"
    return "separated"


def allowed_zones(graph_type="meso"):
    if graph_type == "hard":
        return {"hard_contact"}
    if graph_type == "near":
        return {"hard_contact", "dlvo_zone", "near_gap"}
    if graph_type == "meso":
        return {"hard_contact", "dlvo_zone", "near_gap", "meso_gap"}
    if graph_type == "far":
        return {"hard_contact", "dlvo_zone", "near_gap", "meso_gap", "far_gap"}
```

```
    raise ValueError(f"Unknown graph_type: {graph_type}")


# src/contact_graphs.py
import numpy as np
import networkx as nx
from zones import surface_distance, distance_zone, allowed_zones


def contact_graph(positions, R=500e-9, graph_type="meso", box=None):
    """
    Erzeugt einen Kontaktgraphen aus Positionen.
    Knoten = Protozellen.
    Kanten = Paare, deren Oberflächenabstand in erlaubten Zonen liegt.
    """
    n = len(positions)
    G = nx.Graph()
    G.add_nodes_from(range(n))
    zones_allowed = allowed_zones(graph_type)
    edge_table = []

    for i in range(n):
        for j in range(i + 1, n):
            L = surface_distance(positions[i], positions[j], R=R, box=box)
            zone = distance_zone(L)
            edge_table.append({
                "i": i,
                "j": j,
                "L_m": L,
                "L_nm": L * 1e9,
                "zone": zone,
            })
            if zone in zones_allowed:
                G.add_edge(i, j, L=L, zone=zone)

    return G, edge_table


def graph_features(G):
    components = [len(c) for c in nx.connected_components(G)]
    components_sorted = sorted(components, reverse=True)
    largest = components_sorted[0] if components_sorted else 0
    n = G.number_of_nodes()
    m = G.number_of_edges()
    mean_degree = (2 * m / n) if n > 0 else 0.0

    return {
        "largest_cluster": largest,
        "number_of_components": len(components_sorted),
```

```
        "number_of_edges": m,
        "mean_degree": mean_degree,
        "cluster_size_distribution": components_sorted,
    }

# src/macrostates.py
import networkx as nx


def macrostate_from_graph(G):
    """
    Regelbasierter erster Makrozustandskatalog.
    Der Katalog soll klein bleiben und nur robuste Klassen enthalten.
    """
    if G.number_of_edges() == 0:
        return "monomer_gas"

    components = list(nx.connected_components(G))
    largest_nodes = max(components, key=len)
    largest = len(largest_nodes)
    H = G.subgraph(largest_nodes)

    if largest == 2:
        return "dimer_dominated"

    if largest == 3:
        return "trimer_like"

    if largest == 4:
        # In 2D ist dies nur ein Tetraeder-Proxy.
        if H.number_of_edges() >= 4 and (2 * H.number_of_edges() / 4) >= 2.0:
            return "tetra_like"
        return "loose_four_cluster"

    if largest >= 5:
        return "large_cluster"

    return "mixed_transition"


def macrostate_from_positions(positions, graph_builder, graph_type="meso"):
    G, edges = graph_builder(positions, graph_type=graph_type)
    return macrostate_from_graph(G), G, edges

# src/sequences.py
import numpy as np
import pandas as pd
```

```
from collections import Counter


def dwell_times(sequence):
    if len(sequence) == 0:
        return []
    out = []
    cur = sequence[0]
    length = 1
    for s in sequence[1:]:
        if s == cur:
            length += 1
        else:
            out.append((cur, length))
            cur = s
            length = 1
    out.append((cur, length))
    return out


def filter_short_states(sequence, min_dwell=5):
    """
    Ersetzt kurze Flackerzustände durch den vorherigen validen Zustand.
    Speichern Sie trotzdem immer auch die raw_sequence.
    """
    runs = dwell_times(sequence)
    filtered = []
    previous_valid = None
    for state, length in runs:
        if length >= min_dwell:
            filtered.extend([state] * length)
            previous_valid = state
        else:
            replacement = previous_valid if previous_valid is not None else state
            filtered.extend([replacement] * length)
    return filtered


def transition_matrix(sequence):
    states = sorted(set(sequence))
    counts = pd.DataFrame(0.0, index=states, columns=states)
    for a, b in zip(sequence[:-1], sequence[1:]):
        counts.loc[a, b] += 1.0
    probs = counts.div(counts.sum(axis=1).replace(0, np.nan), axis=0)
    return probs


def return_probability(sequence, target_state, horizon=50):
```

```
    trials = 0
    returns = 0
    for t in range(len(sequence) - 1):
        if sequence[t] == target_state and sequence[t + 1] != target_state:
            trials += 1
            future = sequence[t+1:t+1+horizon]
            if target_state in future:
                returns += 1
    return returns / trials if trials > 0 else np.nan


def mean_return_time(sequence, target_state, horizon=200):
    times = []
    for t in range(len(sequence) - 1):
        if sequence[t] == target_state and sequence[t + 1] != target_state:
            future = sequence[t+1:t+1+horizon]
            for dt, s in enumerate(future, start=1):
                if s == target_state:
                    times.append(dt)
                    break
    return float(np.mean(times)) if times else np.nan

# src/robustness.py
import numpy as np


def jitter_positions(positions, sigma_nm=1.0, seed=0):
    rng = np.random.default_rng(seed)
    sigma = sigma_nm * 1e-9
    return positions + rng.normal(scale=sigma, size=positions.shape)


def label_agreement(seq_a, seq_b):
    n = min(len(seq_a), len(seq_b))
    if n == 0:
        return np.nan
    return sum(seq_a[i] == seq_b[i] for i in range(n)) / n


def robustness_test(position_sequence, label_function, sigma_nm=1.0, seed=0):
    raw_labels = []
    pert_labels = []
    for t, positions in enumerate(position_sequence):
        raw_labels.append(label_function(positions))
        pert = jitter_positions(positions, sigma_nm=sigma_nm, seed=seed + t)
        pert_labels.append(label_function(pert))
    return label_agreement(raw_labels, pert_labels), raw_labels, pert_labels
```

```
# src/run_team2_pipeline.py
import pandas as pd
from contact_graphs import contact_graph, graph_features
from macrostates import macrostate_from_graph
from sequences import filter_short_states, transition_matrix, return_probability,
mean_return_time


def analyze_run(df_run, R=500e-9, graph_type="meso", min_dwell=5):
    """
    Erwartete Spalten:
    run_id, seed, time, particle_id, x, y, optional z
    """
    rows = []
    raw_sequence = []

    for t, frame in df_run.groupby("time"):
        frame = frame.sort_values("particle_id")
        if "z" in frame.columns:
            positions = frame[["x", "y", "z"]].to_numpy()
        else:
            positions = frame[["x", "y"]].to_numpy()

        G, edge_table = contact_graph(positions, R=R, graph_type=graph_type)
        features = graph_features(G)
        macro = macrostate_from_graph(G)
        raw_sequence.append(macro)

        row = {
            "time": t,
            "macrostate_raw": macro,
            **features,
        }
        rows.append(row)

    filtered = filter_short_states(raw_sequence, min_dwell=min_dwell)
    for row, filt in zip(rows, filtered):
        row["macrostate_filtered"] = filt

    result = pd.DataFrame(rows)
    return result


def summarize_sequence(sequence):
    states = sorted(set(sequence))
    rows = []
    for s in states:
        rows.append({
```

```
        "state": s,
        "frequency": sequence.count(s) / len(sequence),
        "return_probability_T50": return_probability(sequence, s, horizon=50),
        "mean_return_time_T200": mean_return_time(sequence, s, horizon=200),
    })
return pd.DataFrame(rows), transition_matrix(sequence)
```

12. Pflichtvisualisierungen

Abbildung	Inhalt	Zweck
Abbildung 1	Zeitreihe raw vs filtered macrostate	zeigt, wie stark Persistenzfilter wirkt
Abbildung 2	Clustergrößenverteilung pro Modell	Vergleich DLVO/vdW/CL
Abbildung 3	Dwell-Time-Verteilungen	Persistenz der Zustände
Abbildung 4	Transition Matrix Heatmap	Dynamik zwischen Makrozuständen
Abbildung 5	Return Probability pro Zustand	Attractor-Kandidaten
Abbildung 6	Label Agreement nach Jitter	Robustheit der Labels
Abbildung 7	Fettsäurevesikel vs PMBC	Materialeffekt auf Makrodynamik

13. Minimal-Deliverables

- Physikalisch begründeter Abstandszonen-Katalog.
- Kontaktgraphen für mindestens drei Graphdefinitionen: hard, near, meso.
- Makrozustandssequenzen raw und filtered.
- Dwell-Time-Verteilungen für alle relevanten Zustände.
- Transition Matrix und Return Probability.
- Label-Robustheit gegen Jitter.
- Vergleich der Modelle vdw_only, dlvo_classic, cl_retarded und optional cl_meso_hypothesis.
- Vergleich Fettsäurevesikel vs PMBC.
- CSV-Export an Team 3.
- Kurze methodische Reflexion: Welche Zustände sind robust, welche Artefakte?

14. Exportformat für Team 3

Spalte	Beschreibung
run_id	eindeutige Simulation
seed	Zufallsseed
time	Zeitschritt
macrostate_raw	ungefiltertes Label
macrostate_filtered	gefiltertes Label für Team 3

model_type	vdw_only, dlvo_classic, cl_retarded, cl_meso_hypothesis
material_type	fatty_acid oder PMBC
A_H	Hamaker-/Materialparameter
salt_mM	Ionenstärke
noise	Rauschparameter
largest_cluster	größte Komponente
mean_degree	mittlerer Kontaktgrad

15. Typische Fehler und Gegenmaßnahmen

Fehler	Gegenmaßnahme
Kontakt binär definieren	Abstandszonen verwenden und Graphvarianten vergleichen
Zu viele Makrozustände	kleiner, stabiler Katalog mit 5–7 Zuständen
Flackerlabels an Team 3 übergeben	raw und filtered speichern; Team 3 nutzt filtered
Tetraeder in 2D überinterpretieren	als tetra_like oder Kompaktheitsproxy deklarieren
Zustände nach Wunschresultaten anpassen	Kriterien vor der Analyse festlegen
Ein Seed reicht nicht	mehrere Seeds pro Modell und Material verwenden
Rückkehr mit Verweildauer verwechseln	Return Probability getrennt von Dwell Time messen

16. Wochenplan

Woche	Aufgaben
1	Datenformat verstehen, Einheiten prüfen, Abstandszonen implementieren
2	Kontaktgraphen und Graphfeatures berechnen; erste Makrozustände labeln
3	Persistenzfilter und Dwell-Time-Analyse implementieren
4	Return Probability, Mean Return Time und Transition Matrix berechnen
5	Robustheit gegen Jitter und Sensitivität gegenüber Kontaktzonen testen
6	Modellvergleich: vdw_only, dlvo_classic, cl_retarded, cl_meso_hypothesis
7	Materialvergleich: Fettsäurevesikel vs PMBC; Export an Team 3 vorbereiten
8	Bericht, Visualisierungen, methodische Reflexion und Endabgabe

17. Bewertungsrubrik

Kriterium	Gewicht	Erwartung
Physikalische Begründung der Abstandszonen	15 %	Zonen sind nachvollziehbar, korrekt und dokumentiert
Implementierung der Kontaktgraphen	15 %	Graphen funktionieren für mehrere Kontaktdefinitionen
Makrozustandskatalog	15 %	klein, robust, nicht überfeinert, interpretierbar
Attractor-Memory-Analyse	20 %	Dwell Times, Return Probability, Transition Matrix sauber berechnet
Robustheit und Sensitivität	15 %	Jitter-Test, Seed-Vergleich, Kontaktzonenvergleich
Übergabequalität an Team 3	10 %	saubere CSVs, Metadaten, filtered sequences
Bericht und Reflexion	10 %	klare Grenzen, keine Overclaims, gute Visualisierungen

18. Finaler Arbeitsauftrag

Entwickeln Sie eine robuste Coarse-Graining-Pipeline, die aus den mikrophysikalischen Simulationen von Team 1 diskrete Makrozustandssequenzen erzeugt. Berechnen Sie dazu physikalisch begründete Abstandszonen, Kontaktgraphen, Graphfeatures und regelbasierte Makrozustände. Filtern Sie kurzlebige Flackerzustände, messen Sie Verweildauern, Rückkehrwahrscheinlichkeiten und Übergangsmatrizen und prüfen Sie die Robustheit der Labels gegen Rauschen. Vergleichen Sie die Makrodynamik für verschiedene physikalische Modelle und Materialien und exportieren Sie eine saubere, reproduzierbare Zustandssequenz an Team 3.

Wichtigster didaktischer Punkt

Ein Makrozustand ist nicht einfach ein Name. Er ist eine wissenschaftliche Entscheidung darüber, welche Mikrokonfigurationen auf der nächsten Beschreibungsebene als gleichwertig gelten.

Gute Makrozustände sind robust, wiedererkennbar, dynamisch relevant und anschlussfähig für ε-Maschinen, Closure, Tasks und Sustainability.